

Práctico 6

Entrada/salida, filesystem y usuarios

Tomás Rodeghiero

Sistemas Operativos — Universidad Nacional de Río Cuarto

2026

Resumen

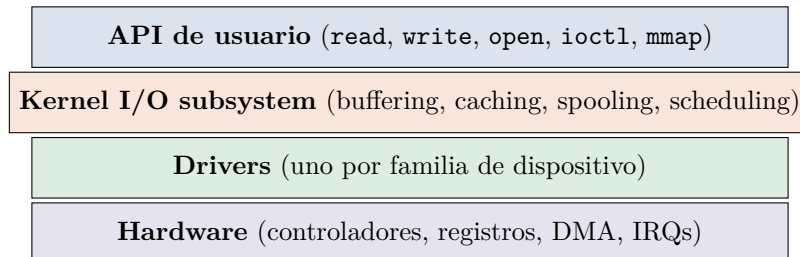
Este documento presenta la resolución integral del Práctico 6 de la materia Sistemas Operativos (UNRC, 2026), centrado en *entrada/salida*, *sistemas de archivos* y *usuarios/control de acceso*. Se resuelven los **diez ejercicios** de la consigna oficial, divididos en dos bloques. El **bloque A** recorre los conceptos de E/S y filesystem: dispositivos de bloque y carácter en `/dev`, uso de `dd` y de los dispositivos especiales `/dev/zero` y `/dev/urandom`, el flujo de compilación y carga de un módulo del kernel (RTC), la representación de un directorio en FAT vs. inodos, y la API `stat/readdir` para navegar el filesystem desde C. El **bloque B** aborda usuarios y permisos: listado de usuarios y grupos en Linux y macOS, semántica de permisos UGO/octal (con énfasis en el bit de ejecución sobre directorios), los `UIDs` real y efectivo, y el funcionamiento del bit `SUID` a través de un ejemplo ejecutable paso a paso. Cada ejercicio se justifica con la teoría del capítulo *Entrada/salida y sistemas de archivos* de las Notas del curso (Marcelo Arroyo).

Índice

1. Introducción	2
Bloque A — Entrada/salida y filesystem	2
2. Ejercicio 1 — Regex para dispositivos de bloque en <code>/dev</code>	2
3. Ejercicio 2 — Archivo de 64 MB lleno de ceros con <code>dd</code>	3
4. Ejercicio 3 — Contraseña <code>base64</code> desde <code>/dev/urandom</code>	5
5. Ejercicio 4 — Compilar y cargar el módulo <code>RTC</code>	6
6. Ejercicio 5 — Filesystem <code>/docs/os-book.pdf</code> : FAT vs. inodos	8
7. Ejercicio 6 — Mostrar tipo y tamaño con <code>stat()</code>	11
Bloque B — Usuarios y control de acceso	13
8. Ejercicio 7 — Listar usuarios y grupos del sistema	13
9. Ejercicio 8 — Permisos UGO y octal sobre archivos y directorios	15
10. Ejercicio 9 — Real <code>UID</code> , Effective <code>UID</code> y shell con <code>getpwuid</code>	18
11. Ejercicio 10 — El bit <code>SUID</code> en acción	19
12. Conclusiones	23

1. Introducción

El subsistema de *entrada/salida* es la capa del SO que abstrae el hardware heterogéneo de los dispositivos y lo presenta como una **API uniforme** para los procesos. La filosofía clásica de UNIX —“todo es un archivo”— llevó esta idea hasta sus últimas consecuencias: discos, teclados, ratones, audio, red, dispositivos especiales (`/dev/zero`, `/dev/urandom`, `/dev/null`) y hasta el propio kernel (`/proc`, `/sys`) se acceden con las mismas syscalls (`open/read/write/close/ioctl`).



Modos de transferencia. El SO puede mover datos entre la memoria y un dispositivo de tres formas:

- **Programmed I/O (PIO):** la CPU mueve cada byte leyendo/escribiendo registros del controlador. Es simple pero gasta toda la CPU del proceso.
- **Interrupt-driven I/O:** la CPU le pide al controlador que haga la operación y se va a hacer otra cosa. Cuando el controlador termina, dispara una *IRQ*.
- **DMA (*Direct Memory Access*):** un controlador independiente mueve datos entre el dispositivo y la RAM sin intervención de la CPU. La CPU solo participa al inicio (configurar la transferencia) y al final (recibir la *IRQ* de terminación).

Importante

DMA **no** libera completamente a la CPU. La CPU sigue siendo quien arranca la transferencia (escribe en los registros del DMA controller) y quien atiende la *IRQ* cuando termina. Lo que se ahorra es el ciclo byte-a-byte intermedio, que es donde está el grueso del trabajo en E/S de bloque.

Tipos de dispositivos.

- **Dispositivos de bloque:** leen y escriben en *bloques* de tamaño fijo (típicamente 512 B o 4 KiB). Permiten acceso aleatorio (`lseek`). Discos, SSDs, particiones. Aparecen con la letra **b** en `ls -l`.
- **Dispositivos de carácter:** leen y escriben byte a byte, normalmente sin posibilidad de `seek`. Teclados, seriales, terminales, `/dev/null`, `/dev/zero`, `/dev/urandom`. Aparecen con **c**.
- **Dispositivos de red:** no aparecen en `/dev`; se acceden con la API de sockets, que es un modelo distinto (`socket`, `bind`, `connect`, ...).

Bloque A — Entrada/salida y filesystem

2. Ejercicio 1 — Regex para dispositivos de bloque en `/dev`

Enunciado. Dar la expresión regular en el comando `ls -l /dev grep "regexp here"|` para que liste *sólo* los dispositivos de bloques.

Análisis

`ls -l` imprime una línea por entrada con el formato

```
<tipo><permisos> <links> <user> <group> <maj>,<min> <fecha> <nombre>
```

El **primer carácter** de cada línea indica el tipo del archivo:

Letra	Tipo	Descripción
-	regular	archivo común (texto, binario, etc.)
d	directorio	contiene entradas (nombre → inodo)
l	symlink	contiene una ruta hacia otro archivo
b	block device	dispositivo accedido en bloques (discos)
c	character device	dispositivo accedido byte a byte (tty, urandom)
p	FIFO / pipe	cola con nombre creada con <code>mkfifo</code>
s	socket	socket de dominio UNIX

Por lo tanto, listar los *dispositivos de bloque* se reduce a filtrar líneas cuyo primer carácter sea b. La expresión correcta es `^b`: el `^` ancla el match al **principio de la línea**.

Solución

```
ls -l /dev | grep "^b"
```

Salida típica en Linux.

```
brw-rw---- 1 root disk 8, 0 ... sda
brw-rw---- 1 root disk 8, 1 ... sda1
brw-rw---- 1 root disk 8, 2 ... sda2
brw-rw---- 1 root cdrom 11, 0 ... sr0
brw-rw---- 1 root disk 7, 0 ... loop0
```

Salida típica en macOS.

```
brw-r----- 1 root operator 1, 0 ... disk0
brw-r----- 1 root operator 1, 1 ... disk0s1
brw-r----- 1 root operator 1, 2 ... disk0s2
```

Idea intuitiva

La línea total `N` que `ls -l` imprime al principio del listado empieza con `t`, así que `^b` ya la descarta. Si se omitiera el ancla `^` y se usara solo `"b"`, también matchearían líneas cuyo nombre contiene una `b` (`sda1` no, pero un dispositivo llamado `cuab0` por ejemplo sí), lo que introduciría falsos positivos. El `^` evita exactamente eso.

Conclusión. La expresión regular pedida es `"^b"`.

3. Ejercicio 2 — Archivo de 64 MB lleno de ceros con `dd`

Enunciado. Crear un archivo de 64 MB inicializado en cero, usando `dd` y leyendo del dispositivo `/dev/zero`.

Análisis

`dd` (*disk dump / data definition*) copia bytes de entrada en bytes de salida con tamaño y cantidad de bloques configurables:

```
dd if=<entrada> of=<salida> bs=<tamano_bloque> count=<nro_bloques>
```

El total escrito es exactamente $bs \times count$. `/dev/zero` es un dispositivo de *carácter* especial que provee bytes nulos (0x00) infinitos: cada `read` sobre él retorna buffers llenos de ceros, sin tocar el disco.

Para escribir 64 MB tenemos varias combinaciones equivalentes:

$$64 \text{ MB} = 64 \times 2^{20} \text{ bytes} = 67\,108\,864 \text{ bytes}$$

bs	count	total
1M	64	64 MB
4K	16 384	64 MB
1024	65 536	64 MB

Idea intuitiva

Cuánto importa elegir `bs`: bloques más grandes significan menos syscalls (una por bloque). En la práctica, copiar 64 MB con `bs=1` (un byte por vez) tarda decenas de segundos, mientras que con `bs=1M` tarda fracciones de segundo. La carga de trabajo no es la transferencia: son las llamadas al sistema y los cambios de contexto.

Solución

La forma idiomática:

```
dd if=/dev/zero of=zero64.img bs=1M count=64
```

Salida esperada.

```
64+0 records in
64+0 records out
67108864 bytes (67 MB, 64 MiB) copied, 0.046 s, 1.5 GB/s
```

Verificación.

```
$ ls -lh zero64.img
-rw-r--r-- 1 tomi tomi 64M ... zero64.img

$ wc -c zero64.img
67108864 zero64.img
```

Importante

Alternativa moderna para crear un archivo de 64 MB de ceros: `truncate -s 64M zero64.img`. La diferencia es que `truncate` crea un *sparse file*: el archivo aparenta tener 64 MB pero no consume bloques en disco hasta que se escriben datos no nulos. Con `dd` se escriben efectivamente 64 MB de ceros físicos al disco.

Conclusión. El comando solicitado es

```
dd if=/dev/zero of=zero64.img bs=1M count=64.
```

4. Ejercicio 3 — Contraseña base64 desde /dev/urandom

Enunciado. Escribir un comando que genere una contraseña de 16 caracteres codificada en base64 desde /dev/urandom, usando `head` y `base64`.

Análisis

Sobre /dev/urandom: es un dispositivo de carácter que entrega bytes pseudoaleatorios del PRNG del kernel. A diferencia de /dev/random, *no se bloquea* cuando el “pool” de entropía se considera bajo; en sistemas modernos esto está considerado seguro para cualquier uso criptográfico de propósito general.

Sobre base64: codifica 3 bytes de entrada en 4 caracteres ASCII del alfabeto {A–Z, a–z, 0–9, +, /}. La relación es:

$$\text{chars_base64} = \lceil \text{bytes_entrada} \times 4 \div 3 \rceil$$

Despejando el camino inverso, para obtener **exactamente** 16 caracteres sin padding (=):

$$\text{bytes_entrada} = 16 \times 3 \div 4 = 12$$

Solución

```
head -c 12 /dev/urandom | base64
```

- `head -c 12 /dev/urandom`: lee exactamente 12 bytes binarios del PRNG.
- Pipe a `base64`: codifica esos 12 bytes en 16 caracteres ASCII.

Ejemplos de salida.

```
$ head -c 12 /dev/urandom | base64
xK9q+Tb2/PfL3aRm
```

```
$ head -c 12 /dev/urandom | base64
hM7Bn5RpQ2Lk8VxC
```

```
$ head -c 12 /dev/urandom | base64
9j+kP/2nXqRtB4vL
```

Idea intuitiva

Cada ejecución produce una contraseña distinta (es lo que se espera de un generador aleatorio). Con 12 bytes aleatorios bien distribuidos hay $2^{12 \times 8} = 2^{96}$ posibilidades, mucho más que suficiente para resistir cualquier ataque por fuerza bruta.

Importante

Para uso real conviene asegurarse de eliminar el newline final que `base64` agrega:

```
head -c 12 /dev/urandom | base64 | tr -d '\n'
```

Y para garantizar exactamente 16 caracteres sin importar padding:

```
head -c 16 /dev/urandom | base64 | head -c 16
```

Conclusión. El comando pedido es
`head -c 12 /dev/urandom |base64|.`

5. Ejercicio 4 — Compilar y cargar el módulo RTC

Enunciado. Compilar el módulo (device driver) Linux RTC (*Real Time Clock*) siguiendo las instrucciones del README. El ejercicio se realiza sobre un sistema Linux con headers del kernel instalados.

Análisis: qué es el RTC y por qué se accede como driver

El **Real Time Clock** es un chip alimentado por batería que mantiene la hora aunque la computadora esté apagada. En arquitectura PC suele estar integrado al chipset (el clásico MC146818 emulado por los SO modernos). El kernel lo expone como dispositivo de carácter en `/dev/rtc0`.

El RTC ofrece tres servicios al SO:

1. Lectura/escritura de la hora actual.
2. Programación de *alarmas* (interrupciones a futuro).
3. Generación de *interrupciones periódicas* (típicamente de 2 Hz a 8192 Hz), útiles para ticks de scheduler en sistemas embebidos.

Anatomía mínima de un módulo del kernel

Un módulo del kernel es código que se compila aparte del kernel principal y se carga/descarga dinámicamente, sin reiniciar. El esqueleto mínimo en C es:

```
/* hello_rtc.c -- esqueleto de un modulo */
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>

static int __init hello_rtc_init(void) {
    printk(KERN_INFO "hello_rtc: modulo cargado\n");
    return 0; /* 0 = éxito, !=0 = falla la carga */
}

static void __exit hello_rtc_exit(void) {
    printk(KERN_INFO "hello_rtc: modulo descargado\n");
}

module_init(hello_rtc_init);
module_exit(hello_rtc_exit);
MODULE_LICENSE("GPL");
MODULE_AUTHOR("Curso SO UNRC");
MODULE_DESCRIPTION("Demo modulo RTC");
```

Diferencias con un programa de usuario:

- No hay `main()`: el “punto de entrada” es la función registrada con `module_init`, que se llama cuando el módulo se carga.
- No se incluye `<stdio.h>`: en el kernel `printf` se reemplaza por `printk`.
- Hay un macro `KERN_INFO` (y otros como `KERN_WARNING`, `KERN_ERR`) que marca el nivel de log del mensaje.
- El módulo se compila con un `Makefile` especial que usa el sistema de *Kbuild* del kernel.

Makefile para compilar el módulo

```
# Makefile
obj-m += hello_rtc.o

KDIR := /lib/modules/$(shell uname -r)/build

all:
    $(MAKE) -C $(KDIR) M=$(PWD) modules

clean:
    $(MAKE) -C $(KDIR) M=$(PWD) clean
```

Importante

La línea `KDIR := /lib/modules/$(shell uname -r)/build` pide los *headers* y scripts del kernel que se está ejecutando. Necesita el paquete `linux-headers-$(uname -r)` (Debian/Ubuntu) o `kernel-devel` (Fedora/RHEL) instalado.

Flujo de compilación, carga y prueba

```
# 1) Compilar
$ make
make -C /lib/modules/6.5.0-15-generic/build M=/home/user/rtc modules
make[1]: Entering directory '/usr/src/linux-headers-6.5.0-15-generic'
CC [M] /home/user/rtc/hello_rtc.o
MODPOST /home/user/rtc/Module.symvers
CC [M] /home/user/rtc/hello_rtc.mod.o
LD [M] /home/user/rtc/hello_rtc.ko

# 2) Cargar el modulo
$ sudo insmod hello_rtc.ko

# 3) Verificar que esta cargado
$ lsmod | grep hello_rtc
hello_rtc 16384 0

# 4) Ver el mensaje del modulo en el log del kernel
$ sudo dmesg | tail -n 2
[12345.678] hello_rtc: modulo cargado

# 5) Descargar
$ sudo rmmod hello_rtc

$ sudo dmesg | tail -n 1
[12346.789] hello_rtc: modulo descargado
```

Leer la hora del RTC desde el espacio de usuario

Una vez cargado el driver del RTC (en general viene en el kernel por defecto), se puede leer la hora con un programa de usuario que abra `/dev/rtc0` y use `ioctl` con el comando `RTC_RD_TIME`:

```
/* read_rtc.c */
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
```

```
#include <sys/ioctl.h>
#include <linux/rtc.h>

int main(void) {
    int fd = open("/dev/rtc0", O_RDONLY);
    if (fd < 0) { perror("open /dev/rtc0"); return 1; }

    struct rtc_time t;
    if (ioctl(fd, RTC_RD_TIME, &t) < 0) {
        perror("ioctl RTC_RD_TIME");
        close(fd);
        return 1;
    }

    printf("Hora del RTC: %04d-%02d-%02d %02d:%02d:%02d\n",
           t.tm_year + 1900, t.tm_mon + 1, t.tm_mday,
           t.tm_hour, t.tm_min, t.tm_sec);

    close(fd);
    return 0;
}
```

```
$ gcc -Wall -o read_rtc read_rtc.c
$ sudo ./read_rtc # /dev/rtc0 generalmente requiere root
Hora del RTC: 2026-03-15 14:23:47
```

Conclusión

El ejercicio ilustra el patrón estándar para extender el kernel con un **driver de dispositivo** como módulo cargable:

1. Código C usando la API del kernel (`printk`, `module_init`/`exit`, etc.).
2. Makefile que delega en Kbuild (`make -C /lib/modules/.../build M=$PWD modules`).
3. Carga/descarga con `insmod`/`rmmod`.
4. Inspección con `lsmod`, `dmesg`, `/proc/modules`.

El RTC, en particular, es accedido desde el espacio de usuario mediante `ioctl` sobre `/dev/rtc0`, ejemplo paradigmático de cómo el SO ofrece una interfaz uniforme (un archivo) para un dispositivo que internamente requiere comandos específicos.

6. Ejercicio 5 — Filesystem `/docs/os-book.pdf`: FAT vs. inodos

Enunciado. Describir un sistema de archivos con la estructura

```
\
|-- docs
    |
    +-- os-book.pdf (2 bloques de datos)
```

usando (a) una representación tipo FAT y (b) una representación basada en *inodos*.

Análisis comparativo

Las dos familias de filesystem responden de forma distinta a la pregunta “dado el nombre de un archivo, ¿cómo encuentro sus bloques?”.

- En **FAT**, la entrada de directorio guarda el *primer cluster* del archivo. Para los demás, se consulta una **tabla centralizada** (la FAT) que es esencialmente una lista enlazada: la entrada `FAT[i]` dice cuál es el próximo cluster del archivo que comenzó en *i*, o un valor especial EOC si *i* es el último.
- En **inodos** (estilo UFS/ext2/...), la entrada de directorio guarda sólo el número de *inodo*. El inodo es un descriptor con toda la metadata del archivo (tamaño, permisos, owner, fechas) y **punteros directos** a sus bloques. La estructura está *descentralizada*: cada archivo lleva su propia lista.

5.a — Representación tipo FAT

Una partición FAT tiene cuatro áreas principales: *boot sector*, *tabla FAT* (a veces duplicada), *directorio raíz* y *área de datos* (clusters numerados a partir de 2, porque 0 y 1 están reservados).

Entradas de directorio. Cada entrada es de tamaño fijo (usualmente 32 bytes) y contiene:

Campo	Significado
<code>name</code>	nombre 8 + 3 chars (FAT16) o de tamaño variable (FAT32 con LFN)
<code>attr</code>	directorio, archivo, oculto, sólo lectura, ...
<code>first_clu</code>	primer cluster del archivo en el área de datos
<code>size</code>	tamaño en bytes (sólo para archivos regulares)

Layout concreto del ejemplo. Supongamos:

- Cluster 3: contenido del directorio `docs`.
- Cluster 5: primer bloque del PDF.
- Cluster 6: segundo bloque del PDF.

Directorio raíz (área especial en FAT16, o cluster fijo en FAT32). Una sola entrada:

name	attr	first_clu	size
DOCS	DIR	3	0

Contenido del cluster 3 (directorio `docs`):

name	attr	first_clu	size
.	DIR	3	0
..	DIR	0 (raíz)	0
OS-BOOK PDF	—	5	2×bs

Tabla FAT. Cada entrada `FAT[i]` dice quién sigue al cluster *i*:

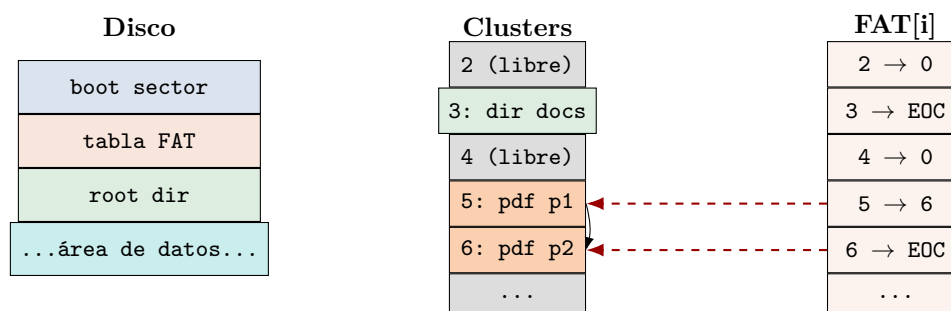
i	FAT[i]
0	reservado (marcador)
1	reservado
2	libre (0)
3 (raíz de docs)	EOC
4	libre
5 (os-book.pdf parte 1)	6
6 (os-book.pdf parte 2)	EOC
≥ 7	libre

Idea intuitiva

Para leer `/docs/os-book.pdf` el SO recorre:

- (1) Raíz: encuentra entrada `DOCS`, `first_clu = 3`.
- (2) Va al cluster 3, encuentra entrada `OS-BOOK.PDF`, `first_clu = 5`.
- (3) Lee el cluster 5.
- (4) Consulta `FAT[5] = 6` → lee el cluster 6.
- (5) Consulta `FAT[6] = EOC` → termina.

Diagrama del layout FAT



5.b — Representación con inodos

En un FS basado en inodos cada archivo o directorio tiene *un* inodo numerado de forma única dentro del FS. El inodo guarda toda la metadata y los punteros a bloques de datos. Las entradas de directorio sólo asocian nombres con números de inodo.

Estructura típica de un inodo (estilo UFS clásico):

Campo	Significado
<code>i_mode</code>	tipo (regular, dir, link, ...) y permisos
<code>i_uid, i_gid</code>	owner y grupo
<code>i_size</code>	tamaño en bytes
<code>i_atime, mtime, ctime</code>	timestamps de acceso, modificación, cambio de inodo
<code>i_blocks</code>	bloques físicos usados
<code>i_direct[0..11]</code>	punteros directos a bloques de datos
<code>i_indirect1</code>	puntero indirecto simple
<code>i_indirect2</code>	puntero indirecto doble
<code>i_indirect3</code>	puntero indirecto triple

Asignación de inodos al ejemplo.

# inodo	Quién es	Apunta a (bloques de datos)
1	/ (raíz, directorio)	B_{root}
2	/docs (directorio)	B_{docs}
3	/docs/os-book.pdf (regular)	$direct[0]=B_{pdf1}, direct[1]=B_{pdf2}$

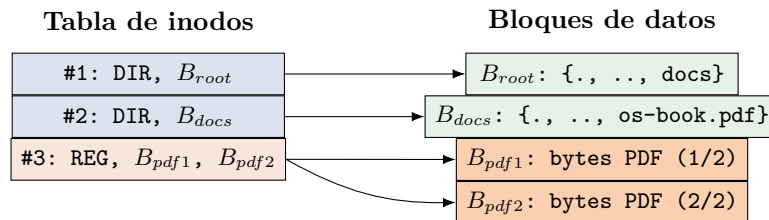
Contenido de los bloques de directorio (cada entrada = nombre → inodo):

- B_{root} : { `.` → 1, `..` → 1, `docs` → 2 }
- B_{docs} : { `.` → 2, `..` → 1, `os-book.pdf` → 3 }
- B_{pdf1}, B_{pdf2} : bytes binarios del PDF.

Notas.

- La raíz tiene siempre el mismo número de inodo (en ext2/3/4 el #2, no el #1, que se usa para tracking de bloques dañados; el #1 usado aquí es ilustrativo).
- .. de la raíz apunta a la raíz misma.
- `i_size` del inodo 3 es $\leq 2 \times \text{bs}$ (no necesariamente exactamente 2 bloques: el archivo puede terminar a mitad del segundo).

Diagrama del layout con inodos



Conclusión: contraste FAT vs. inodos

	FAT	Inodos
Metadata del archivo	en la entrada de directorio	en el inodo
Lista de bloques	FAT centralizada (lista enlazada)	punteros en el inodo
Hard links (varios nombres)	No soportado	Sí: varias entradas → mismo inodo
Acceso aleatorio rápido	no (recorrer FAT)	sí (punteros directos / indirectos)
Robustez ante FAT corrupta	toda la partición se pierde	sólo se afecta un archivo

Resumen. La diferencia esencial es *dónde* se guarda la información de qué bloques pertenecen a qué archivo. En FAT, toda la información de pertenencia está en una sola tabla compartida. En inodos, cada archivo lleva su propia “tabla” (los punteros del inodo), y los directorios se reducen a una asociación de nombres con números de inodo.

7. Ejercicio 6 — Mostrar tipo y tamaño con `stat()`

Enunciado. El siguiente programa lee el directorio corriente e imprime el nombre y el inodo de cada entrada. Extenderlo para mostrar también el **tipo** (regular, directorio, link, ...) y el **tamaño** de cada archivo, usando `stat()` sobre `st_mode` y `st_size`.

Análisis: por qué necesitamos `stat()`

`readdir()` devuelve una `struct dirent` con el *nombre* y el *número de inodo* de la entrada, pero **no** con la metadata completa (tipo, tamaño, permisos, fechas). Esa información vive en el inodo, no en la entrada de directorio.

Para obtenerla hay que invocar `stat()` (o sus variantes `lstat`, `fstat`), que consulta el inodo del archivo y llena una `struct stat` con todos los campos:

```

struct stat {
    dev_t st_dev; /* dispositivo */
    ino_t st_ino; /* numero de inodo */
    mode_t st_mode; /* tipo + permisos */
    nlink_t st_nlink; /* cantidad de hard links */
    uid_t st_uid; /* owner */
    gid_t st_gid; /* grupo */
    off_t st_size; /* tamaño en bytes */
    blksize_t st_blksize; /* tamaño del bloque */
    blkcnt_t st_blocks; /* bloques fisicos asignados */
    time_t st_atime, st_mtime, st_ctime;
  }
  
```

```
/* ... */
};
```

Tipo del archivo en `st_mode`. El campo combina dos informaciones: tipo y permisos. El tipo se extrae con macros `S_IS<Tipo>(m)` provistas por `<sys/stat.h>`:

Macro	Devuelve no-cero si...
<code>S_ISREG(m)</code>	el archivo es regular
<code>S_ISDIR(m)</code>	el archivo es un directorio
<code>S_ISLNK(m)</code>	el archivo es un symlink
<code>S_ISCHR(m)</code>	el archivo es un character device
<code>S_ISBLK(m)</code>	el archivo es un block device
<code>S_ISFIFO(m)</code>	el archivo es una FIFO / pipe con nombre
<code>S_ISSOCK(m)</code>	el archivo es un socket de UNIX

Importante

stat vs lstat. Si la entrada de directorio es un **symlink**, `stat()` sigue el link y reporta los datos del *archivo apuntado*, no del link mismo. Para obtener los datos del link en sí, hay que usar `lstat()`. En este programa conviene `lstat()` para no confundir symlinks con sus targets.

Solución

```
#include <stdio.h>
#include <string.h>
#include <dirent.h>
#include <sys/stat.h>

static const char *file_type(mode_t m) {
    if (S_ISREG(m)) return "REG ";
    if (S_ISDIR(m)) return "DIR ";
    if (S_ISLNK(m)) return "LNK ";
    if (S_ISCHR(m)) return "CHR ";
    if (S_ISBLK(m)) return "BLK ";
    if (S_ISFIFO(m)) return "FIFO";
    if (S_ISSOCK(m)) return "SOCK";
    return "? ";
}

int main(void) {
    DIR *dir = opendir(".");
    if (!dir) { perror("opendir"); return 1; }

    struct dirent *entry;
    while ((entry = readdir(dir)) != NULL) {
        struct stat st;
        if (lstat(entry->d_name, &st) < 0) {
            perror(entry->d_name);
            continue;
        }
        printf("%-25s inode=%-10lu type=%s size=%lld bytes\n",
            entry->d_name,
            (unsigned long)entry->d_ino,
            file_type(st.st_mode),
            (long long)st.st_size);
    }
    closedir(dir);
    return 0;
}
```

```
}
```

Salida típica

Ejecutándolo en un directorio con un poco de variedad:

```
$ gcc -Wall -o lsdire lsdire.c
$ ./lsdire
. inode=14385 type=DIR size=4096 bytes
.. inode=14112 type=DIR size=4096 bytes
lsdire.c inode=14502 type=REG size=812 bytes
lsdire inode=14503 type=REG size=16400 bytes
notas.txt inode=14504 type=REG size=125 bytes
backup -> notas.txt inode=14505 type=LNK size=10 bytes
```

Lectura de los resultados

- **Directorios** (`.`, `..`): el campo `size` es el tamaño *del bloque de directorio* (en ext4 típicamente 4096 B), NO la suma del contenido. Para saber cuánto pesan los archivos de adentro hay que recorrer el directorio recursivamente (lo que hace `du`).
- **Archivos regulares**: `size` es el tamaño en bytes del contenido.
- **Symlinks**: `size` con `lsstat` es la *longitud del target string* (en este caso 10, los chars de "notas.txt"). Con `stat`, en cambio, habría dado 125, el tamaño del archivo apuntado.

Conclusión. Las dos pistas claves para resolver el ejercicio fueron: (1) usar `stat/lsstat` para obtener metadata adicional sobre cada entrada de directorio, y (2) decodificar `st_mode` con las macros `S_IS<Tipo>(m)` para mapear el bitfield a una etiqueta legible.

Bloque B — Usuarios y control de acceso

8. Ejercicio 7 — Listar usuarios y grupos del sistema

Enunciado. Listar los usuarios y grupos del sistema. En macOS, usar el comando `dsc1` (*directory services command-line*).

Análisis

Un sistema UNIX mantiene la información de cuentas en dos archivos históricos:

- `/etc/passwd`: una línea por usuario, 7 campos separados por `:`.
- `/etc/group`: una línea por grupo, 4 campos.

En sistemas modernos puede no usarse este archivo directamente (por ejemplo, cuando hay LDAP, Active Directory u Open Directory en macOS), pero la API portable sigue siendo la misma: `getpwent`, `getgrent` en C, o `getent` en shell.

Linux

Usuarios.

```
$ cat /etc/passwd | head -n 4
root:x:0:0:root:/root:/bin/bash
```

```
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin

# Forma portable: consulta la base de datos del sistema
$ getent passwd | wc -l
38
```

Formato de /etc/passwd.

#	Campo	Significado
1	name	login del usuario
2	passwd	contraseña (x significa “en /etc/shadow”)
3	uid	identificador numérico
4	gid	grupo primario
5	gecos	información extendida (nombre real, etc.)
6	home	directorio personal
7	shell	shell de login (/bin/bash, /usr/sbin/nologin, ...)

Grupos.

```
$ cat /etc/group | head -n 4
root:x:0:
daemon:x:1:
bin:x:2:
sys:x:3:

$ getent group sudo
sudo:x:27:tomi,otra_persona
```

Formato de /etc/group. name : passwd : gid : members_separados_por_coma.

macOS

macOS no usa /etc/passwd para usuarios reales (sólo para algunas cuentas del sistema). La fuente de verdad es **Open Directory**, accesible vía **dscl**:

```
# Listar todos los usuarios
$ dscl . -list /Users
_amavisd
_appleevents
_appowner
...
root
tomi

# Filtrar solo los "humanos" (los del sistema empiezan con _)
$ dscl . -list /Users | grep -v '^_'
daemon
nobody
root
tomi

# Listar todos los grupos
$ dscl . -list /Groups | head
_amavisd
_appleevents
```

```
...
# Detalle de un usuario
$ dscl . -read /Users/tomi
NFSHomeDirectory: /Users/tomi
PrimaryGroupID: 20
RealName: Tomas Rodeghiero
RecordName: tomi
UniqueID: 501
UserShell: /bin/zsh
```

Idea intuitiva

Convención macOS: los usuarios y grupos del *sistema* (no destinados a humanos) empiezan con `_`. Filtrar con `grep -v '^_'` muestra sólo cuentas regulares.

Conclusión. En Linux la forma portable es `getent passwd/getent group` (consulta cualquier backend configurado en `/etc/nsswitch.conf`); en macOS, el comando equivalente es `dscl . -list /Users` y `dscl . -list /Groups`.

9. Ejercicio 8 — Permisos UGO y octal sobre archivos y directorios

Enunciado. Crear un directorio dentro del directorio de trabajo. Analizar los permisos asignados, qué significa el bit de ejecución sobre un directorio, crear un archivo dentro con `touch`, manipular sus permisos con UGO y octal, y observar el efecto.

Análisis: permisos en UNIX

Cada archivo tiene una tripla de permisos:

Categoría	Letra	Quién
Owner	u	el dueño del archivo
Group	g	los miembros del grupo del archivo
Other	o	cualquier otro usuario
All	a	u + g + o (atajo)

Y tres bits por cada categoría:

Bit	Sobre archivo	Sobre directorio
r (4)	leer contenido	listar nombres (<code>ls</code>)
w (2)	modificar contenido	crear/borrar/renombrar entradas
x (1)	ejecutar como programa	atravesar (<code>cd</code> , acceder a entradas por nombre)

8.a — Permisos por defecto y el bit x en directorios

```
$ mkdir mydir
$ ls -ld mydir
drwxr-xr-x 2 tomi staff 64 ... mydir
```

Lectura de `drwxr-xr-x`.

Posición	d	rwX	r-X	r-X
Significado	directorio	owner: lee, escribe, atraviesa	group: lee, atraviesa	other: lee, atraviesa
Octal	—	7	5	5

Es decir, 755. ¿Por qué 755 y no 777? Por el *umask*, una máscara de bits que el shell aplica al permiso “base” (777 para directorios). El umask típico es 022:

$$\text{permiso_final} = 0777 \text{ AND NOT umask} = 0777 \text{ AND NOT } 022 = 755$$

¿Qué significa el bit **x** en un directorio?

- **Con x**: se puede “entrar” al directorio con `cd`, abrir archivos por su nombre y atravesarlo como parte de una ruta más larga.
- **Sin x**: aun teniendo **r** se pueden *listar nombres* (`ls mydir`) pero *no acceder al contenido* (`ls -l` falla porque necesita leer el inodo de cada entrada, lo cual requiere atravesar).
- **Con r y sin x**: ver nombres pero no acceder.
- **Con x y sin r**: acceder a archivos cuyos nombres se conozcan de antemano, pero no listar.

Idea intuitiva

La pareja **r/x** sobre directorios es notoria por confundir: **r** es “ver la guía telefónica” (listar entradas) y **x** es “llamar a un número conocido” (acceder a un archivo). Son ortogonales.

8.b — Crear un archivo con `touch` y mirar permisos

```
$ cd mydir
$ touch test.txt
$ ls -l test.txt
-rw-r--r-- 1 tomi staff 0 ... test.txt
```

Permisos = 644.

$$\text{permiso_final} = 0666 \text{ AND NOT } 022 = 0644$$

(Para archivos regulares la base es 666, no 777: **los archivos no nacen con x**; hay que ponerlo explícitamente con `chmod +x`.)

8.c — Quitar el permiso de lectura

Sintaxis UGO (modificación simbólica relativa):

```
$ chmod a-r test.txt # 'a' = u+g+o, '-r' quita el bit r
$ ls -l test.txt
--w----- 1 tomi staff 0 ... test.txt
```

Sintaxis octal (especificación absoluta):

```
$ chmod 200 test.txt # 200 = u=w, g=---, o=---
$ ls -l test.txt
--w----- 1 tomi staff 0 ... test.txt
```

Las dos son equivalentes. UGO es más explícita en *qué cambia*; octal es más explícita en *cómo queda al final*.

Octal	Binario	UGO	Permiso
7	111	rw x	lee + escribe + ejecuta
6	110	rw -	lee + escribe
5	101	r -x	lee + ejecuta
4	100	r -	lee
3	011	- w x	escribe + ejecuta
2	010	- w -	escribe
1	001	-x	ejecuta
0	000	--	nada

8.d — Intentar editar el archivo

Con 200 el dueño tiene sólo **w**. Veamos qué pasa:

```
# Leer falla
$ cat test.txt
cat: test.txt: Permission denied

# Editar con vim/nano: depende del editor
$ vim test.txt
"test.txt" Permission denied
Press ENTER or type command to continue

# Sobrescribir con redireccion: FUNCIONA, porque solo necesita 'w'
$ echo "hola" > test.txt

# Append: tambien funciona (solo 'w', no 'r')
$ echo "otra linea" >> test.txt
```

Importante

Editar con un editor no es lo mismo que escribir. Los editores como **vim**, **nano** o **emacs** necesitan *leer* el contenido actual antes de mostrarlo. Sin **r**, fallan al abrirlo. Pero el shell puede sobrescribir un archivo con **>** usando solo el bit **w**. Es una distinción importante: “no poder editar” es más restrictivo que “no poder leer”.

8.e — Restaurar la lectura sólo al dueño

UGO:

```
$ chmod u+r test.txt # agrega r solo al owner
$ ls -l test.txt
-rw----- 1 tomi staff ...
```

Octal equivalente:

```
$ chmod 600 test.txt
```

8.f — Volver a editar

Con 600 el dueño tiene **rw**, así que cualquier editor abre el archivo sin problemas:

```
$ vim test.txt # abre normalmente, edita, guarda
```

Conclusión. Los puntos clave del ejercicio:

1. Los permisos por defecto los fija el `umask` (típicamente 022): 755 para directorios, 644 para archivos.
2. El bit `x` en un directorio es “permiso para atravesar”, no “permiso para ejecutar”.
3. UGO y octal son dos formas de la misma operación; cada bit corresponde a una posición en el octal ($r = 4, w = 2, x = 1$).
4. Sin `r` se puede aún *sobrescribir* con `>` pero no abrir con un editor; los editores necesitan ambas cosas.

10. Ejercicio 9 — Real UID, Effective UID y shell con `getpwuid`

Enunciado. Sobre el programa dado, que imprime el real UID y el effective UID con su nombre asociado vía `getpwuid()`, **extenderlo** para que también muestre el *shell* que usa el real user.

Análisis: real UID vs. effective UID

Todo proceso UNIX tiene cuatro identificadores de usuario asociados:

ID	Significado
<code>ruid</code> (real)	quién lanz ó el proceso. Casi siempre fijo durante la vida del proceso.
<code>euid</code> (effective)	quién es “a los efectos de chequear permisos” en cada sys-call.
<code>suid</code> (saved-set)	valor previo del <code>euid</code> ; permite swap entre <code>ruid</code> / <code>euid</code> .
<code>fsuid</code> (filesystem)	sólo Linux: <code>euid</code> a efectos de operaciones de FS.

En la mayoría de los procesos `ruid = euid = suid`. La distinción se vuelve relevante con programas SUID/SGID (Ejercicio 10).

`getpwuid(uid)` consulta la base de datos de usuarios (`/etc/passwd` o equivalente) por el UID dado y devuelve una `struct passwd *`:

```
struct passwd {
    char *pw_name; /* login name */
    char *pw_passwd; /* hashed password */
    uid_t pw_uid; /* user id */
    gid_t pw_gid; /* group id */
    char *pw_gecos; /* descripcion */
    char *pw_dir; /* home directory */
    char *pw_shell; /* default shell */
};
```

Solución

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <pwd.h>

int main(void) {
    uid_t ruid = getuid(); /* real UID */
    uid_t euid = geteuid(); /* effective UID */

    struct passwd *pw_r = getpwuid(ruid);
    if (pw_r == NULL) { perror("getpwuid(ruid)"); return 1; }
```

```

printf("Process Real User ID: %d, name: %s\n", ruid, pw_r->pw_name);
printf("Real user shell : %s\n", pw_r->pw_shell); /* <-- agregado */

struct passwd *pw_e = getpwuid(euid);
if (pw_e == NULL) { perror("getpwuid(euid)"); return 1; }

printf("Process Effective User ID: %d, name: %s\n", euid, pw_e->pw_name);

return 0;
}

```

Compilación y ejecución

```

$ gcc -Wall -o whoami_extended whoami_extended.c
$ ./whoami_extended
Process Real User ID: 1000, name: tomi
Real user shell : /bin/bash
Process Effective User ID: 1000, name: tomi

```

Observación. En una ejecución normal `ruid = euid`. La diferencia aparecerá en el Ejercicio 10 cuando se active el bit SUID.

Idea intuitiva

El campo `pw_shell` es el *login shell* configurado para el usuario (séptimo campo de `/etc/passwd`). No siempre es el shell que el usuario está usando ahora: si abrió una sub-sesión con `zsh`, el valor de `pw_shell` sigue mostrando el shell por defecto. Para conocer el shell en uso *en este preciso proceso* hay que consultar la variable de ambiente `$SHELL` del entorno heredado.

Conclusión. La extensión es de una sola línea: imprimir `pw_r->pw_shell` después de obtener el `struct passwd` del real UID. Pero el ejercicio fuerza a entender la diferencia entre el UID *real* (quién lanzó el proceso) y el *efectivo* (con qué identidad se evalúan los permisos en cada syscall).

11. Ejercicio 10 — El bit SUID en acción

Enunciado. Extender el programa anterior para que (a) abra el archivo creado en el ejercicio 8 para lectura/escritura, (b) se cambie el dueño del archivo a `root` y se vea que falla, (c) se ejecute con `sudo` y se vea que funciona, (d) se cambie el dueño del propio programa a `root` y se vea qué pasa, (e) se encienda el bit SUID y se ejecute como usuario regular — explicando en cada paso por qué.

Análisis: qué hace el bit SUID

Cuando un usuario ejecuta un binario, el SO crea un proceso cuyo `ruid` y `euid` son ambos iguales al UID del usuario. Pero si el binario tiene el bit **SUID** encendido, el SO asigna al proceso un *euid igual al dueño del binario*. En símbolos, al hacer `exec` de un programa con SUID:

```

ruid_nuevo = ruid_actual
euid_nuevo = owner_del_programa

```

Importante

SUID es el mecanismo histórico de UNIX para permitir que un programa sin privilegios delegue una acción privilegiada al kernel a través de un binario que pertenece a un usuario privilegiado. El ejemplo canónico es `/usr/bin/passwd`, que pertenece a `root` con SUID encendido: lo ejecuta cualquier usuario pero corre como `root`, lo cual le permite escribir en `/etc/shadow`.

Programa extendido

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>
#include <pwd.h>

int main(void) {
    uid_t ruid = getuid();
    uid_t euid = geteuid();
    struct passwd *pw_r = getpwuid(ruid);
    struct passwd *pw_e = getpwuid(euid);

    printf("ruid=%d (%s), euid=%d (%s)\n",
           ruid, pw_r ? pw_r->pw_name : "?",
           euid, pw_e ? pw_e->pw_name : "?");

    /* Abrir el archivo creado en el ejercicio 8 */
    int fd = open("test.txt", O_RDWR);
    if (fd < 0) {
        perror("open test.txt");
        return 1;
    }
    printf("Archivo abierto OK (fd=%d)\n", fd);
    close(fd);
    return 0;
}
```

Compilación:

```
$ gcc -Wall -o whoami_open whoami_open.c
```

Estado inicial: `test.txt` es del usuario regular con permisos 600, y `whoami_open` también es del usuario regular con permisos 755.

10.a — Abrir el archivo: OK

```
$ ./whoami_open
ruid=1000 (tomi), euid=1000 (tomi)
Archivo abierto OK (fd=3)
```

Por qué funciona. El `euid` del proceso es 1000 (`tomi`), el archivo es de `tomi` con permisos 600 (`rw` para el dueño). El kernel chequea “`euid == st_uid && (mode & S_IWUSR)`” y deja pasar.

10.b — Cambiar el dueño del archivo a `root`

```
$ sudo chown root test.txt
$ ls -l test.txt
-rw----- 1 root staff 10 ... test.txt
```

10.c — Repetir 10.a: ahora falla

```
$ ./whoami_open
ruid=1000 (tomi), euid=1000 (tomi)
open test.txt: Permission denied
```

Por qué falla. El euid sigue siendo tomi, pero el archivo es de root con permisos 600 (sólo el dueño puede leer/escribir). El kernel rechaza con EACCES (Permission denied).

10.d — Ejecutar con sudo: funciona

```
$ sudo ./whoami_open
ruid=0 (root), euid=0 (root)
Archivo abierto OK (fd=3)
```

Por qué funciona. sudo ejecuta el comando con ruid = euid = 0 (root). El archivo es de root, así que el chequeo del kernel pasa.

Idea intuitiva

sudo es a su vez un binario **SUID-root**: pertenece a root con bit SUID encendido. Por eso cualquier usuario lo ejecuta y se le otorga (temporalmente, y previa autenticación) la identidad de root. Lo que estamos viendo en 10.d es la consecuencia de un programa SUID-root muy famoso.

10.e — Cambiar el dueño del programa a root

```
$ sudo chown root whoami_open
$ ls -l whoami_open
-rwxr-xr-x 1 root staff 16400 ... whoami_open
```

El programa ahora es de root, pero conserva permisos 755, así que el usuario regular sigue pudiendo ejecutarlo.

10.f — Ejecutar sin sudo: sigue fallando

```
$ ./whoami_open
ruid=1000 (tomi), euid=1000 (tomi)
open test.txt: Permission denied
```

Por qué falla. Aunque el *programa* sea de root, al ejecutarlo el sistema le asigna al proceso un euid igual al del usuario que lo invocó (tomi). Cambiar el dueño del programa por sí solo *no* cambia el euid del proceso.

10.g — Con sudo: vuelve a funcionar

```
$ sudo ./whoami_open
ruid=0 (root), euid=0 (root)
Archivo abierto OK (fd=3)
```

Misma razón que 10.d.

10.h — Encender el bit SUID en el programa

```
$ sudo chmod u+s whoami_open
$ ls -l whoami_open
-rwsr-xr-x 1 root staff 16400 ... whoami_open
```

Lectura del nuevo permiso. La letra **s** en lugar de **x** en la posición del owner indica que el bit SUID está encendido. (Si el archivo tuviera SUID pero no **x**, aparecería una **S** mayúscula.)

En octal, SUID es el bit 4000:

$$4755 = \text{SUID} + \text{rwxr-xr-x}$$

10.i — Ejecutar como usuario regular: ahora funciona

```
$ ./whoami_open
ruid=1000 (tomi), euid=0 (root) <- !!!
Archivo abierto OK (fd=3)
```

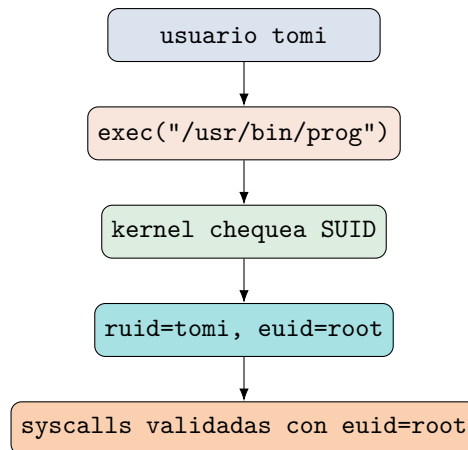
Por qué funciona. El kernel, al hacer **exec** del binario SUID:

- Setea **ruid** = 1000 (el que lanzó el proceso).
- **Setea euid** = 0 (el dueño del programa).

A partir de ahí, cuando el programa llama **open("test.txt", O_RDWR)**, el kernel chequea con **euid = 0**: pasa el chequeo porque **root** puede abrir cualquier cosa.

Resumen y diagrama del juego de UIDs

Paso	Dueño test.txt	Dueño prog	SUID?	ejec. con sudo?	resultado
10.a	tomi	tomi	no	no	OK
10.c	root	tomi	no	no	EACCES
10.d	root	tomi	no	sí	OK
10.f	root	root	no	no	EACCES
10.g	root	root	no	sí	OK
10.i	root	root	sí	no	OK



Importante

SUID es peligroso. Un programa SUID-root con un bug que permita ejecución arbitraria (buffer overflow, inyección de comandos, race condition) le da **root** al atacante. Por eso los binarios SUID-root del sistema son extremadamente auditados (**passwd**, **sudo**, **su**, **ping** con sockets raw, etc.) y se reducen al mínimo posible. Encontrarlos en un sistema:

```
$ find / -perm -u+s -type f 2>/dev/null
```

Conclusión del ejercicio. La secuencia ilustra los tres elementos que el kernel mira al chequear permisos sobre un archivo: (1) el **euid** del proceso (no el **ruid**), (2) el dueño y los permisos del archivo (no del programa), y (3) si el binario tiene el bit SUID, qué hace al **exec** (setear **euid** al dueño del binario). Estos tres puntos en interacción son la base de la seguridad clásica de UNIX.

12. Conclusiones

El práctico recorrió, ejercicio a ejercicio, las dos grandes capas del subsistema de E/S que ofrece UNIX al programador:

- Los **Ejercicios 1 a 4** se concentraron en el *lado bajo*, donde los dispositivos físicos aparecen como archivos especiales en **/dev**. El primero ejercitó la identificación de tipos de dispositivo en la salida de **ls**; el segundo y el tercero usaron pseudo-dispositivos (**/dev/zero** y **/dev/urandom**) como fuentes de datos; el cuarto exploró el flujo completo de construcción de un *driver* como módulo cargable del kernel, con su comunicación con el RTC vía **ioctl** sobre **/dev/rtc0**.
- Los **Ejercicios 5 y 6** subieron a la abstracción del *filesystem*: el 5 contrastó las dos representaciones canónicas (FAT centralizada vs. inodos descentralizados) y dejó en claro cuál es la diferencia conceptual (dónde vive la metadata y la cadena de bloques de cada archivo); el 6 mostró el patrón **readdir + stat** para inspeccionar entradas de un directorio desde C.
- Los **Ejercicios 7 a 10** cubrieron usuarios y control de acceso. El 7 fue el lado descriptivo (cómo el sistema guarda y expone usuarios y grupos en Linux y macOS); el 8, la mecánica concreta de los permisos UGO y octal sobre archivos y directorios, con énfasis en el bit **x** en directorios; el 9, la distinción entre **ruid** y **euid**; y el 10, el clímax del bloque: el bit **SUID**, que es la pieza que hace posible que un proceso no privilegiado obtenga, durante la ejecución de un binario cuidadosamente diseñado, los privilegios del dueño de ese binario.

Idea intuitiva

La idea que une todo el práctico es que **en UNIX, todo es un archivo**, y **para todo archivo hay un dueño y permisos**. La combinación de esas dos abstracciones simples — inodo + permisos UGO— es lo que permite que el mismo subsistema sirva indistintamente para acceder a discos, dispositivos, sockets, pipes y memoria compartida, con un control de acceso uniforme.